

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: C

Batch: 3

Practical No: 2

Roll no: 59

PRN No: 12413745

Name of Student: Netra Lalwani

Subject Name: Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm:

Multilayer Perceptron (MLP) for Iris/Wine Dataset Classification

1. **Import** the required libraries.
2. **Load** the Iris (or Wine) dataset.
3. **Split** the dataset into training and testing sets.
4. **Create** an MLP classifier by specifying:
 - Hidden layers
 - Activation function (ReLU)
 - Optimizer (Adam)
 - Number of iterations
5. **Train** the MLP model using the training data.
6. **Predict** the class labels for the test data.
7. **Calculate** the accuracy of the model using the predicted and actual labels.
8. **Generate** the confusion matrix to compare actual and predicted classes.
9. **Display** the accuracy, confusion matrix, and classification report.
10. **Analyse** the performance of the MLP classifier.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, roc_curve, auc
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

plt.rcParams["font.family"] = "DejaVu Sans"
plt.rcParams["axes.edgecolor"] = "#333333"
sns.set_style("whitegrid")

data = load_wine()
X, y = data.data, data.target
class_names = data.target_names
feature_names = data.feature_names

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(64, 32, 16), activation="relu", solver="adam",
                    alpha=1e-4, learning_rate_init=1e-3, max_iter=2000, early_stopping=True,
                    validation_fraction=0.15, n_iter_no_change=25, random_state=42)
mlp.fit(X_train_s, y_train)

y_pred = mlp.predict(X_test_s)
y_proba = mlp.predict_proba(X_test_s)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=class_names)

print(f"Test Accuracy: {acc * 100:.2f}%")
print(report)

pca = PCA(n_components=2, random_state=42)
X_train_pca = pca.fit_transform(X_train_s)
X_test_pca = pca.transform(X_test_s)

mlp_2d = MLPClassifier(hidden_layer_sizes=(64, 32, 16), activation="relu", solver="adam",
                       alpha=1e-4, max_iter=2000, early_stopping=True, random_state=42)
mlp_2d.fit(X_train_pca, y_train)
```

```

xx, yy = np.meshgrid(np.linspace(X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1, 300),
                    np.linspace(X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1, 300))
Z = mlp_2d.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
fpr, tpr, roc_auc = {}, {}, {}
for i in range(3):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

activation = X_test_s
for layer in mlp.coefs_[:-1]:
    pass
hidden_out = X_test_s
for i in range(len(mlp.coefs_) - 1):
    hidden_out = np.maximum(0, hidden_out @ mlp.coefs_[i] + mlp.intercepts_[i])
tsne = TSNE(n_components=2, random_state=42, perplexity=25)
hidden_tsne = tsne.fit_transform(hidden_out)

fig = plt.figure(figsize=(20, 16), facecolor="#0f1117")
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.4, wspace=0.35)
palette = ["#00d4ff", "#ff5f6d", "#ffc93c"]
cmap_bg = sns.color_palette(["#003844", "#4a0e1e", "#4d3b00"], as_cmap=False)

def style_ax(ax):
    ax.set_facecolor("#161925")
    ax.tick_params(colors="#cccccc")
    for spine in ax.spines.values():
        spine.set_color("#333c4d")
    ax.title.set_color("#ffffff")
    ax.xaxis.label.set_color("#cccccc")
    ax.yaxis.label.set_color("#cccccc")

ax1 = fig.add_subplot(gs[0, 0])
style_ax(ax1)
sns.heatmap(cm, annot=True, fmt="d", cmap="mako", cbar=False, xticklabels=class_names,
            yticklabels=class_names, ax=ax1, annot_kws={"color": "white", "size": 13})
ax1.set_title(f"Confusion Matrix (Acc: {acc*100:.2f}%)", fontsize=13, fontweight="bold")
ax1.set_xlabel("Predicted")
ax1.set_ylabel("Actual")

ax2 = fig.add_subplot(gs[0, 1])
style_ax(ax2)
ax2.plot(mlp.loss_curve_, color="#00d4ff", linewidth=2)
ax2.set_title("Training Loss Curve", fontsize=13, fontweight="bold")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Loss")
ax2.grid(alpha=0.15)

```

```

ax3 = fig.add_subplot(gs[0, 2])
style_ax(ax3)
if hasattr(mlp, "validation_scores_") and mlp.validation_scores_:
    ax3.plot(mlp.validation_scores_, color="#ffc93c", linewidth=2)
    ax3.set_title("Validation Accuracy per Epoch", fontsize=13, fontweight="bold")
    ax3.set_xlabel("Epoch")
    ax3.set_ylabel("Val Accuracy")
else:
    ax3.axis("off")

ax4 = fig.add_subplot(gs[1, :2])
style_ax(ax4)
ax4.contourf(xx, yy, Z, alpha=0.35, colors=palette)
for i, cname in enumerate(class_names):
    mask = y_train == i
    ax4.scatter(X_train_pca[mask, 0], X_train_pca[mask, 1], color=palette[i], label=cname,
                edgecolor="white", linewidth=0.4, s=45, alpha=0.9)
ax4.set_title("Decision Boundary (PCA-reduced 2D Feature Space)", fontsize=13, fontweight="bold")
ax4.set_xlabel("Principal Component 1")
ax4.set_ylabel("Principal Component 2")
ax4.legend(facecolor="#161925", labelcolor="white", framealpha=0.6)

ax5 = fig.add_subplot(gs[1, 2])
style_ax(ax5)
for i, cname in enumerate(class_names):
    ax5.plot(fpr[i], tpr[i], color=palette[i], linewidth=2, label=f"{cname} (AUC={roc_auc[i]:.2f})")
ax5.plot([0, 1], [0, 1], linestyle="--", color="#666666")
ax5.set_title("Multiclass ROC Curves", fontsize=13, fontweight="bold")
ax5.set_xlabel("False Positive Rate")
ax5.set_ylabel("True Positive Rate")
ax5.legend(facecolor="#161925", labelcolor="white", fontsize=8, framealpha=0.6)

ax6 = fig.add_subplot(gs[2, 0])
style_ax(ax6)
for i, cname in enumerate(class_names):
    mask = y_test == i
    ax6.scatter(hidden_tsne[mask, 0], hidden_tsne[mask, 1], color=palette[i], label=cname,
                edgecolor="white", linewidth=0.4, s=45, alpha=0.9)
ax6.set_title("t-SNE of Learned Hidden Representations", fontsize=13, fontweight="bold")
ax6.set_xlabel("t-SNE 1")
ax6.set_ylabel("t-SNE 2")
ax6.legend(facecolor="#161925", labelcolor="white", framealpha=0.6)

```

```

ax7 = fig.add_subplot(gs[2, 1])
style_ax(ax7)
importances = np.abs(mlp.coefs_[0]).sum(axis=1)
order = np.argsort(importances)[::-1][:8]
ax7.barh(np.array(feature_names)[order][::-1], importances[order][::-1], color="#00d4ff")
ax7.set_title("Top Input Feature Weights (Layer 1)", fontsize=13, fontweight="bold")
ax7.set_xlabel("Summed Absolute Weight")

ax8 = fig.add_subplot(gs[2, 2])
style_ax(ax8)
per_class_acc = cm.diagonal() / cm.sum(axis=1)
ax8.bar(class_names, per_class_acc, color=palette)
ax8.set_ylim(0, 1.05)
ax8.set_title("Per-Class Accuracy", fontsize=13, fontweight="bold")
ax8.set_ylabel("Accuracy")
for i, v in enumerate(per_class_acc):
    ax8.text(i, v + 0.02, f"{v*100:.1f}%", ha="center", color="white", fontsize=10)

```

Output:

Test Accuracy: 97.78%				
	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	15
class_1	0.95	1.00	0.97	18
class_2	1.00	0.92	0.96	12
accuracy			0.98	45
macro avg	0.98	0.97	0.98	45
weighted avg	0.98	0.98	0.98	45



